

Blockchain

For the Academic year 2025 – 2026

INDEX

Sr. No.	Title	Date	Signature
1.	Write the following programs for Blockchain in Python: A) A simple client class that generates the private and public keys by using the built in Python RSA algorithm and test it. B) A transaction class to send and receive money and test it. C) Create multiple transactions and display them. D) Create a blockchain, a genesis block and execute it. E) Create a mining function and test it. F) Add blocks to the miner and dump the blockchain.		
2.	Install and configure Go Ethereum and the Mist browser. Develop and test a sample application.		
3.	Implement and demonstrate the use of the following in Solidity: A) Variable, Operators, Loops, Decision Making, Strings, Arrays, Enums, Structs, Mappings, Conversions, Ether Units, Special Variables. B) Functions, Function Modifiers, View functions, Pure Functions, Fallback Function, Function Overloading, Mathematical functions, Cryptographic functions.		
4.	Implement and demonstrate the use of the following in Solidity: A) Withdrawal Pattern, Restricted Access. B) Contracts, Inheritance, Constructors, Abstract Contracts, Interfaces. C) Libraries, Assembly, Events, Error handling.		
5.	Install hyperledger fabric and composer. Deploy and execute the application.		
6.	Write a program to demonstrate mining of Ether.		
7.	Demonstrate the running of the blockchain node.		
8.	Demonstrate the use of Bitcoin Core API.		
9.	Create your own blockchain and demonstrate its use.		
10.	Build Dapps with angular.		

Practical 1A

Aim: A simple client class that generates the private and public keys by using the built-in Python RSA algorithm and test it.

!pip3 install Crypto

!pip3 install pycrypto

```
import hashlib
import random
import binascii
import datetime
import collections
from Crypto.PublicKey import RSA
from Crypto import Random
from Crypto.Cipher import PKCS1_v1_5

class Client:
    def __init__(self):

        random = Random.new().read
        self._private_key = RSA.generate(1024, random)
        self._public_key = self._private_key.publickey()
        self._signer = PKCS1_v1_5.new(self._private_key)

    @property
    def identity(self):
        return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')

Dinesh = Client()
print ("sender ",Dinesh.identity)
```

Output:

sender 30819f300d06092a864886f.....

Practical 1B

Aim: Create a transaction class to send and receive money and test it.

following imports are required by PKI

import hashlib

import binascii

import datetime

import collections

from Crypto.PublicKey import RSA

from Crypto import Random

from Crypto.Cipher import PKCS1_v1_5

from collections import OrderedDict

import Crypto

import Crypto.Random

from Crypto.Hash import SHA

from Crypto.Signature import PKCS1_v1_5

class Client:

def __init__(self):

random = Random.new().read

self._private_key = RSA.generate(1024, random)

self._public_key = self._private_key.publickey()

self._signer = PKCS1_v1_5.new(self._private_key)

@property

def identity(self):

return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')

class Transaction:

def __init__(self, sender, recipient, value):

self.sender = sender

self.recipient = recipient

self.value = value

self.time = datetime.datetime.now()

def to_dict(self):

if self.sender == "Genesis":

identity = "Genesis"

else:

identity = self.sender.identity

return collections.OrderedDict({

'sender': identity,

```
'recipient': self.recipient,
'value': self.value,
'time' : self.time})

def sign_transaction(self):
    private_key = self.sender._private_key
    signer = PKCS1_v1_5.new(private_key)
    h = SHA.new(str(self.to_dict()).encode('utf8'))
    return binascii.hexlify(signer.sign(h)).decode('ascii')

def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()
    print ("sender: " + dict['sender'])
    print ('-----')
    print ("recipient: " + dict['recipient'])
    print ('-----')
    print ("value: " + str(dict['value']))
    print ('-----')
    print ("time: " + str(dict['time']))
    print ('-----')

transactions = []

Dinesh = Client()
Ramesh = Client()
Suresh = Client()

t1 = Transaction(
    Dinesh,
    Ramesh.identity,
    15.0
)

t1.sign_transaction()
transactions.append(t1)

t2 = Transaction(
    Ramesh,
    Suresh.identity,
    25.0
)

t2.sign_transaction()
transactions.append(t2)
```

```
t3 = Transaction(  
    Ramesh,  
    Suresh.identity,  
    200.0  
)  
  
tn=1  
for t in transactions:  
    print("Transaction #",tn)  
    display_transaction (t)  
    tn=tn+1  
    print ('-----')
```

Output:

```
Transaction # 1  
sender: 30819f300d060...  
-----  
recipient: 30819f300d02a864....  
-----  
value: 15.0  
-----  
time: 2022-04-26 04:07:59.162213  
-----  
-----  
Transaction # 2  
sender: 30819f300d06092a8.....  
-----  
recipient: 30819f300d06092a8.....  
-----  
value: 25.0  
-----  
time: 2022-04-26 04:07:59.165396  
-----  
-----  
Transaction # 3  
sender: 30819f300d06092a8648....  
-----  
recipient: 30819f300d06092a86488...  
-----  
value: 200.0  
-----  
time: 2022-04-26 04:07:59.168579  
-----  
-----
```

Practical 1C

Aim: Create multiple transactions and display them

```
import hashlib
import binascii
import datetime
import collections

from Crypto.PublicKey import RSA
from Crypto import Random
from Crypto.Cipher import PKCS1_v1_5
from collections import OrderedDict
import Crypto
import Crypto.Random
from Crypto.Hash import SHA
from Crypto.Signature import PKCS1_v1_5
class Client:
    def __init__(self):
        random = Random.new().read
        self._private_key = RSA.generate(1024, random)
        self._public_key = self._private_key.publickey()
        self._signer = PKCS1_v1_5.new(self._private_key)
    @property
    def identity(self):
        return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')

class Transaction:
    def __init__(self, sender, recipient, value):
        self.sender = sender
        self.recipient = recipient
        self.value = value
        self.time = datetime.datetime.now()

    def to_dict(self):
        if self.sender == "Genesis":
            identity = "Genesis"
        else:
            identity = self.sender.identity

        return collections.OrderedDict({
            'sender': identity,
            'recipient': self.recipient,
            'value': self.value,
```

```
'time' : self.time}})

def sign_transaction(self):
    private_key = self.sender._private_key
    signer = PKCS1_v1_5.new(private_key)
    h = SHA.new(str(self.to_dict()).encode('utf8'))
    return binascii.hexlify(signer.sign(h)).decode('ascii')

def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()
    print ("sender: " + dict['sender'])
    print ('-----')
    print ("recipient: " + dict['recipient'])
    print ('-----')
    print ("value: " + str(dict['value']))
    print ('-----')
    print ("time: " + str(dict['time']))
    print ('-----')

transactions = []

A = Client()
B = Client()
t1 = Transaction(
    A,
    B.identity,
    15.0
)
t1.sign_transaction()
display_transaction (t1)

#####

Output:

sender: 30819f300d0609.....
-----
recipient: 30819f300d06.....
-----
value: 15.0
-----
time: 2022-04-26 04:00:21.070283
-----
```

Practical 1D

Aim: Create a blockchain, a genesis block and execute it.

following imports are required by PKI

```
import hashlib
import binascii
import datetime
import collections
```

```
from Crypto.PublicKey import RSA
from Crypto import Random
from Crypto.Cipher import PKCS1_v1_5
from collections import OrderedDict
import Crypto
import Crypto.Random
from Crypto.Hash import SHA
from Crypto.Signature import PKCS1_v1_5
```

```
class Client:
```

```
    def __init__(self):
        random = Random.new().read
        self._private_key = RSA.generate(1024, random)
        self._public_key = self._private_key.publickey()
        self._signer = PKCS1_v1_5.new(self._private_key)
    @property
    def identity(self):
        return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
```

```
class Transaction:
```

```
    def __init__(self, sender, recipient, value):
        self.sender = sender
        self.recipient = recipient
        self.value = value
        self.time = datetime.datetime.now()

    def to_dict(self):
        if self.sender == "Genesis":
            identity = "Genesis"
        else:
            identity = self.sender.identity
```



```
    return collections.OrderedDict({
        'sender': identity,
        'recipient': self.recipient,
        'value': self.value,
        'time' : self.time})

def sign_transaction(self):
    private_key = self.sender._private_key
    signer = PKCS1_v1_5.new(private_key)
    h = SHA.new(str(self.to_dict()).encode('utf8'))
    return binascii.hexlify(signer.sign(h)).decode('ascii')

def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()
    print ("sender: " + dict['sender'])
    print ('-----')
    print ("recipient: " + dict['recipient'])
    print ('-----')
    print ("value: " + str(dict['value']))
    print ('-----')
    print ("time: " + str(dict['time']))
    print ('-----')

def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')

class Block:
    def __init__(self):
        self.verified_transactions = []
        self.previous_block_hash = ""
        self.Nonce = ""

Dinesh = Client()

t0 = Transaction (
    "Genesis",
```

```
Dinesh.identity,  
500.0  
)  
  
block0 = Block()  
block0.previous_block_hash = None  
Nonce = None  
block0.verified_transactions.append (t0)  
digest = hash (block0)  
last_block_hash = digest  
  
TPCoins = []  
TPCoins.append (block0)  
  
dump_blockchain(TPCoins)
```

Output:

```
Number of blocks in the chain: 1  
block # 0  
sender: Genesis  
-----  
recipient: 30819f300d06092.....  
-----  
value: 500.0  
-----  
time: 2022-04-26 04:24:05.232662
```

Practical 1E**Aim: Create a mining function and test it.**

```
import hashlib

def sha256(message):
    return hashlib.sha256(message.encode('ascii')).hexdigest()

def mine(message, difficulty=1):
    assert difficulty >= 1
    #if(difficulty <1):
    #    return
    #'1'*2=> '11'
    prefix = '1' * difficulty
    print("prefix",prefix)
    for i in range(1000):
        digest = sha256(str(hash(message)) + str(i))
        print("testing=>" + digest)
        if digest.startswith(prefix):
            print ("after " + str(i) + " iterations found nonce: " + digest)
            return i #i= nonce value

mine ("test message",2)
```

#####

Output:

```
prefix 11
testing=>ab7d1f2b4ba63486a274d7a8c5e4dde793c2d47069ae19ab832dc1177622a182
testing=>cf0a36c4f0c3107cba7a8ebe690db004a01f659bc0aed3b327f01fab0065bf41
testing=>fb0eac040f5f40cd4a39373ca0e6165c07a36db3df510b4c0ad4d45654caeabb
testing=>a298e97de6df74e3856aabb5aed9807652d98a9911a6431bdb3bad0ad2a7bd
testing=>7ff8aa3e5b40e1b5bed59ab464c9b98ceff64b2445cc446cc89ecd93330cba1e
.....
testing=>1cddb5b7e9af6eda960e734606c33f0ce676a7e557a22ba4d7b9af557b0c0360
testing=>29d2f56130e7b276b3cfb94687ff3b1d5c79b6dc8238fe259aaef5af19fd8b2
testing=>3a5f4dcfed5301f36be80fd7d42573b1585ea4ef9037e96853affe66d68f8a04
testing=>ddb4d9dc8c7f20443eedc9ac798aebb2c080cc46926dc0151760e37097bf2dcf
testing=>4fb1010880723ce012526941ae6236260852c8e995583d0d2f65b6f9ff655c61
testing=>11038c5fc4f90108f4198097c76c9af5d38c92b48fe27968eacbd89324fe9d2a
after 21 iterations found nonce:
11038c5fc4f90108f4198097c76c9af5d38c92b48fe27968eacbd89324fe9d2a
21
```

Practical 1F**Aim: Add blocks to the miner and dump the blockchain.**

following imports are required by PKI

import hashlib

import random

import binascii

import datetime

import collections

from Crypto.PublicKey import RSA

from Crypto import Random

from Crypto.Cipher import PKCS1_v1_5

from collections import OrderedDict

import Crypto

import Crypto.Random

from Crypto.Hash import SHA

from Crypto.Signature import PKCS1_v1_5

class Client:

def __init__(self):

random = Random.new().read

self._private_key = RSA.generate(1024, random)

self._public_key = self._private_key.publickey()

self._signer = PKCS1_v1_5.new(self._private_key)

@property

def identity(self):

return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')

class Transaction:

def __init__(self, sender, recipient, value):

self.sender = sender

self.recipient = recipient

self.value = value

self.time = datetime.datetime.now()

def to_dict(self):

if self.sender == "Genesis":

identity = "Genesis"

else:

identity = self.sender.identity

return collections.OrderedDict({

```
'sender': identity,
'recipient': self.recipient,
'value': self.value,
'time' : self.time})

def sign_transaction(self):
    private_key = self.sender._private_key
    signer = PKCS1_v1_5.new(private_key)
    h = SHA.new(str(self.to_dict()).encode('utf8'))
    return binascii.hexlify(signer.sign(h)).decode('ascii')

def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()
    print ("sender: " + dict['sender'])
    print ('-----')
    print ("recipient: " + dict['recipient'])
    print ('-----')
    print ("value: " + str(dict['value']))
    print ('-----')
    print ("time: " + str(dict['time']))
    print ('-----')

def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')

class Block:
    def __init__(self):
        self.verified_transactions = []
        self.previous_block_hash = ""
        self.Nonce = ""

def sha256(message):
    return hashlib.sha256(message.encode('ascii')).hexdigest()

def mine(message, difficulty=1):
    assert difficulty >= 1
```

```
#if(difficulty <1):  
#    return  
# '1'*3=> '111'  
prefix = '1' * difficulty  
for i in range(1000):  
    digest = sha256(str(hash(message)) + str(i))  
    if digest.startswith(prefix):  
        return i #i= nonce value
```

```
A = Client()  
B = Client()  
C = Client()  
t0 = Transaction (  
    "Genesis",  
    A.identity,  
    500.0  
)
```

```
t1 = Transaction (  
    A,  
    B.identity,  
    40.0  
)
```

```
t2 = Transaction (  
    A,  
    C.identity,  
    70.0  
)
```

```
t3 = Transaction (  
    B,  
    C.identity,  
    700.0  
)
```

```
#blockchain
```

```
TPCoins = []
```

```
block0 = Block()  
block0.previous_block_hash = None  
Nonce = None  
block0.verified_transactions.append (t0)  
digest = hash (block0)  
last_block_hash = digest #last_block_hash it is hash of block0  
TPCoins.append (block0)
```

```
block1 = Block()
block1.previous_block_hash = last_block_hash
block1.verified_transactions.append(t1)
block1.verified_transactions.append(t2)
block1.Nonce=mine(block1, 2)
digest = hash(block1)
last_block_hash = digest
TPCoins.append(block1)
```

```
block2 = Block()
block2.previous_block_hash = last_block_hash
block2.verified_transactions.append(t3)
Nonce = mine(block2, 2)
block2.Nonce=mine(block2, 2)
digest = hash(block2)
last_block_hash = digest
TPCoins.append(block2)
```

```
dump_blockchain(TPCoins)
```

```
#####
```

Output:

Number of blocks in the chain: 3

block # 0

sender: Genesis

recipient: 30819f300d0609.....

value: 500.0

time: 2022-04-26 04:30:59.070952

=====

block # 1

sender: 30819f300d06092a86.....

recipient: 30819f300d06092a.....

value: 40.0

time: 2022-04-26 04:30:59.071076

sender: 30819f300d06092a86....

recipient: 30819f300d06092a....

value: 70.0

time: 2022-04-26 04:30:59.071174

=====

block # 2

sender: 30819f300d06092a....

recipient: 30819f300d06092a....

value: 700.0

time: 2022-04-26 04:30:59.071272

=====

Practical 3A

AIM: WRITE A SOLIDITY PROGRAM FOR VARIABLES, OPERATORS, LOOPS, DECISION MAKING, STRING, ARRAYS, ENUMS, STRUCTS, MAPPINGS, CONVERSIONS, ETHER UNITS, SPECIAL VARIABLES.

A)Variables:

supports three types of variables.

State Variables – Variables whose values are permanently stored in a contract storage.

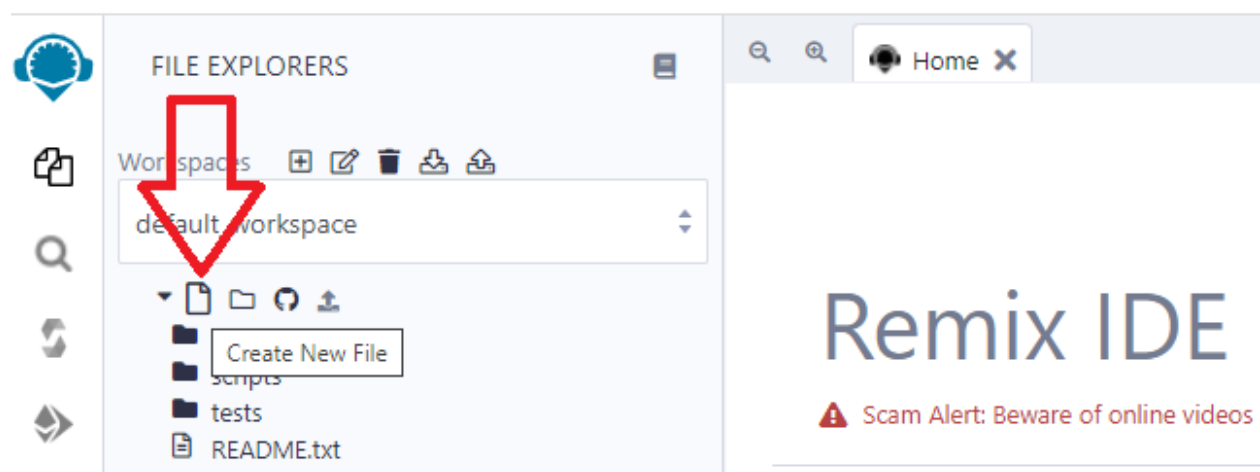
Local Variables – Variables whose values are present till function is executing.

Global Variables – Special variables exists in the global namespace used to get information about the blockchain.i.e. `blockhash(uint blockNumber)` returns (bytes32), `block.coinbase` (address payable), `block.difficulty` (uint)....and many more

Step 1: Open this website

<https://remix.ethereum.org/>

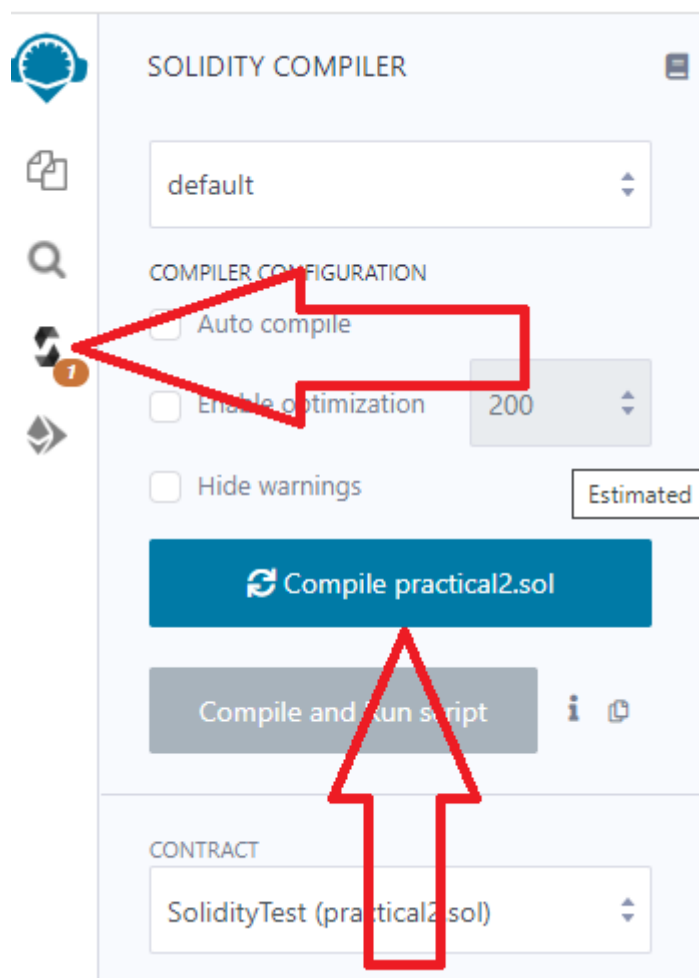
Step 2: Create new file – practical.sol



Step 3: Write this program in the new file

```
//////////  
pragma solidity ^0.5.0;  
contract SolidityTest {  
    uint storedData; // State variable  
    constructor() public {  
        storedData = 10;  
    }  
    function getResult() public view returns(uint){  
        uint a = 1; // local variable  
        uint b = 2;  
        uint result = a + b;  
        return result; //access the state variable  
    }  
}
```

Step 4: Compile contract



Step 5: Deploy contract

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT

JavaScript VM (London)

VM

ACCOUNT +

0x5B3...eddC4 (100 ether)

GAS LIMIT

3000000

VALUE

0 Wei

CONTRACT

SolidityTest - practical2.sol

Deploy

Step 6: Select the contract and click button

DEPLOY & RUN TRANSACTIONS

CONTRACT

SolidityTest - practical2.sol

Deploy

☐ Publish to IPFS

OR

At Address

Transactions recorded **1**

Deployed Contracts

▼ SOLIDITYTEST AT 0XD91...39138 (MEM)

getResult

0: uint256: 3

```
1 pragma solidity ^0.4.18;
2 contract Sol {
3     uint storedData;
4     constructor(uint value) public {
5         storedData = value;
6     }
7     function getResult() public returns (uint) {
8         uint a = 1;
9         uint b = 2;
10        uint result = a + b;
11        return result;
12    }
13 }
14
```

1.State Variable:

```
// Solidity program to
// demonstrate state
// variables
pragma solidity ^0.5.0;
// Creating a contract
contract Solidity_var_Test {
// Declaring a state variable
uint8 public state_var;
// Defining a constructor
constructor() public {
state_var = 16;
}
}
```



Transactions recorded 3



Deployed Contracts



▼ SOLIDITY_VAR_TEST AT 0XF8E...9FBE8 (I



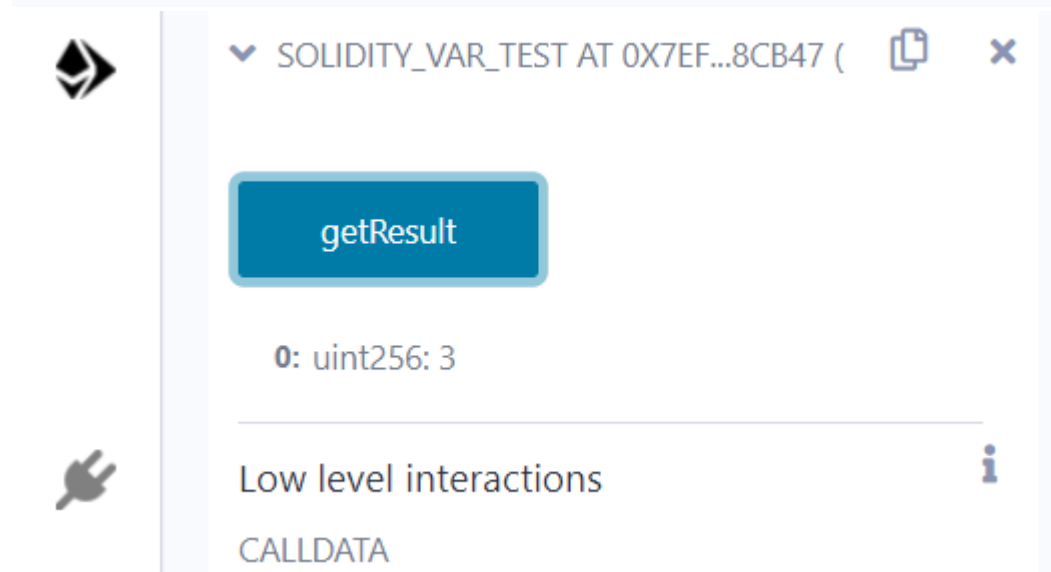
state_var

0: uint8: 16

2.Local Variable:

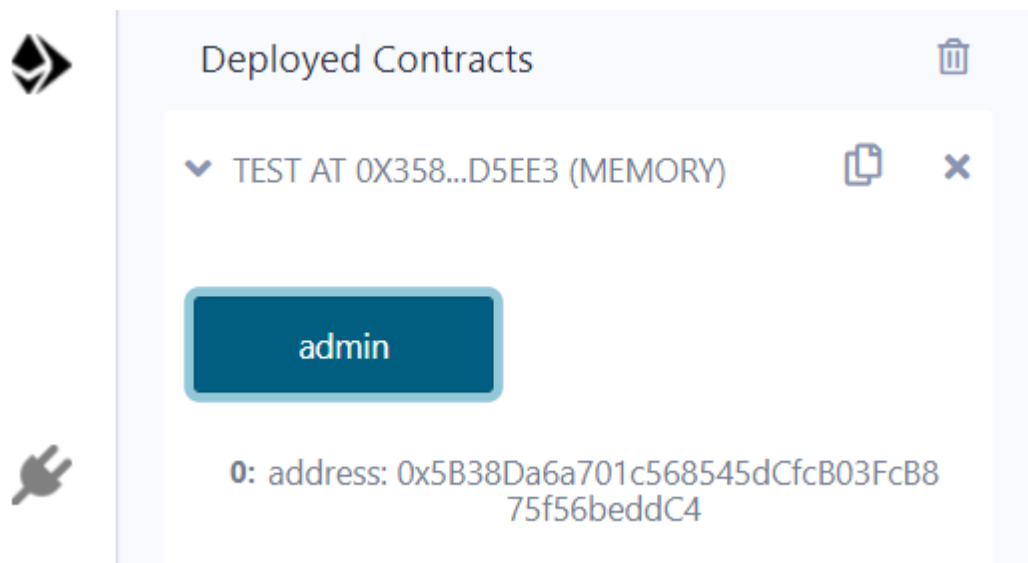
```
// Solidity program to demonstrate
// local variables
pragma solidity ^0.5.0;
// Creating a contract
contract Solidity_var_Test {
// Defining function to show the declaration and
```

```
// scope of local variables
function getResult() public view returns(uint){
// Initializing local variables
uint local_var1 = 1;
uint local_var2 = 2;
uint result = local_var1 + local_var2;
// Access the local variable
return result;
}
}
```



3.Global variable:

```
// Solidity program to
// show Global variables
pragma solidity ^0.5.0;
// Creating a contract
contract Test {
// Defining a variable
address public admin;
// Creating a constructor to
// use Global variable
constructor() public {
admin = msg.sender;
}
}
```



Scope of local variables is limited to function in which they are defined but State variables can have three types of scopes.

Public – Public state variables can be accessed internally as well as via messages. For a public state variable, an automatic getter function is generated.

Internal – Internal state variables can be accessed only internally from the current contract or contract deriving from it without using this.

Private – Private state variables can be accessed only internally from the current contract they are defined not in the derived contract from it.

B)Operators

Solidity supports the following types of operators.

Arithmetic Operators

Comparison Operators

Logical (or Relational) Operators

Assignment Operators

Conditional (or ternary) Operators

1. Arithmetic Operator

// Solidity contract to demonstrate

// Arithmetic Operator

```
pragma solidity ^0.5.0;
// Creating a contract
contract SolidityTest {
// Initializing variables
uint16 public a = 20;
uint16 public b = 10;
// Initializing a variable
// with sum
uint public sum = a + b;
// Initializing a variable
// with the difference
uint public diff = a - b;
// Initializing a variable
// with product
uint public mul = a * b;
// Initializing a variable
// with quotient
uint public div = a / b;
// Initializing a variable
// with modulus
uint public mod = a % b;
// Initializing a variable
// decrement value
uint public dec = --b;
// Initializing a variable
// with increment value
uint public inc = ++a;
}
```




2.Relational Operator

// Solidity program to demonstrate

// Relational Operator

```
pragma solidity ^0.5.0;
```

// Creating a contract

```
contract SolidityTest {
```

// Declaring variables

```
uint16 public a = 20;
```

```
uint16 public b = 10;
```

// Initializing a variable

// with bool equal result

```
bool public eq = a == b;
```

// Initializing a variable

```
// with bool not equal result
bool public noteq = a != b;

// Initializing a variable
// with bool greater than result
bool public gtr = a > b;

// Initializing a variable
// with bool less than result
bool public les = a < b;

// Initializing a variable
// with bool greater than equal to result
bool public gtreq = a >= b;

// Initializing a variable
// bool less than equal to result
bool public leseq = a <= b;
}
```

3.Logical Operators

```
// Solidity program to demonstrate
```

```
// Logical Operators
```

```
pragma solidity ^0.5.0;

// Creating a contract
contract logicalOperator{

// Defining function to demonstrate
// Logical operator
function Logic(
bool a, bool b) public view returns(
bool, bool, bool){

// Logical AND operator
bool and = a&&b;

// Logical OR operator
bool or = a||b;
```

```
// Logical NOT operator
```

```
bool not = !a;
```

```
return (and, or, not);
```

```
}
```

```
}
```

4.Bitwise Operators

```
// Solidity program to demonstrate
```

```
// Bitwise Operator
```

```
pragma solidity ^0.5.0;
```

```
// Creating a contract
```

```
contract SolidityTest {
```

```
// Declaring variables
```

```
uint16 public a = 20;
```

```
uint16 public b = 10;
```

```
// Initializing a variable
```

```
// to '&' value
```

```
uint16 public and = a & b;
```

```
// Initializing a variable
```

```
// to '|' value
```

```
uint16 public or = a | b;
```

```
// Initializing a variable
```

```
// to '^' value
```

```
uint16 public xor = a ^ b;
```

```
// Initializing a variable
```

```
// to '<<' value
```

```
uint16 public leftshift = a << b;
```

```
// Initializing a variable
```

```
// to '>>' value
```

```
uint16 public rightshift = a >> b;
```

```
// Initializing a variable
```

```
// to '~' value
```

```
uint16 public not = ~a ;  
  
}
```

5.Assignment Operator

// Solidity program to demonstrate

// Assignment Operator

```
pragma solidity ^0.5.0;  
  
// Creating a contract  
contract SolidityTest {  
  
    // Declaring variables  
    uint16 public assignment = 20;  
    uint public assignment_add = 50;  
    uint public assign_sub = 50;  
    uint public assign_mul = 10;  
    uint public assign_div = 50;  
    uint public assign_mod = 32;  
  
    // Defining function to  
    // demonstrate Assignment Operator  
    function getResult() public {  
        assignment_add += 10;  
        assign_sub -= 20;  
        assign_mul *= 10;  
        assign_div /= 10;  
        assign_mod %= 20;  
        return ;  
    }  
}
```

6.Conditional Operators

// Solidity program to demonstrate

// Conditional Operator

```
pragma solidity ^0.5.0;

// Creating a contract
contract SolidityTest{
// Defining function to demonstrate
// conditional operator
function sub(
uint a, uint b) public view returns(
uint){
uint result = (a > b? a-b : b-a);
return result;
}
}
```

C)Loops:

1.While loop: The most basic loop in Solidity is the **while** loop which would be discussed in this chapter. The purpose of a **while** loop is to execute a statement or code block repeatedly as long as an **expression** is true. Once the expression becomes **false**, the loop terminates.

2.do-while loop: The **do...while** loop is similar to the **while** loop except that the condition check happens at the end of the loop. This means that the loop will always be executed at least once, even if the condition is **false**.

3.for loop: The **for** loop is the most compact form of looping. It includes the following three important parts –

The **loop initialization** where we initialize our counter to a starting value. The initialization statement is executed before the loop begins.

The **test statement** which will test if a given condition is true or not. If the condition is true, then the code given inside the loop will be executed, otherwise the control will come out of the loop.

The **iteration statement** where you can increase or decrease your counter.

4.loop control: Solidity provides full control to handle loops and switch statements. There may be a situation when you need to come out of a loop without reaching its bottom. There may also be a situation when you want to skip a part of your code block and start the next iteration of the loop.To handle all such situations, Solidity

provides **break** and **continue** statements. These statements are used to immediately come out of any loop or to start the next iteration of any loop respectively.

1.While Loop

```
pragma solidity ^0.5.0;
contract SolidityTest {
    uint storedData;
    constructor() public {
        storedData = 10;
    }
    function getResult() public view returns(string memory){
        uint a = 10;
        uint b = 2;
        uint result = a + b;
        return integerToString(result);
    }
    function integerToString(uint _i) internal pure
    returns (string memory) {
        if(_i == 0) {
            return "0";
        }
        uint j = _i;
        uint len;
        while (j != 0) {
            len++;
            j /= 10;
        }
        bytes memory bstr = new bytes(len);
        uint k = len - 1;
        while (_i != 0) { // while loop
            bstr[k--] = byte(uint8(48 + _i % 10));
            _i /= 10;
        }
        return string(bstr);
    }
}
```

2.Do-while loop:

```
pragma solidity ^0.5.0;
contract SolidityTest {
    uint storedData;
    constructor() public {
        storedData = 10;
    }
    function getResult() public view returns(string memory){
        uint a = 10;
        uint b = 2;
        uint result = a + b;
        return integerToString(result);
    }
    function integerToString(uint _i) internal pure
    returns (string memory) {
        if (_i == 0) {
            return "0";
        }
        uint j = _i;
        uint len;
        while (j != 0) {
            len++;
            j /= 10;
        }
        bytes memory bstr = new bytes(len);
        uint k = len - 1;
        do {           // do while loop
            bstr[k--] = byte(uint8(48 + _i % 10));
            _i /= 10;
        }
        while (_i != 0);
        return string(bstr);
    }
}
```

3.For Loop:

```
pragma solidity ^0.5.0;

contract SolidityTest {
    uint storedData;
    constructor() public {
        storedData = 10;
    }

    function getResult() public view returns(string memory){
        uint a = 10;
        uint b = 2;
        uint result = a + b;
        return integerToString(result);
    }

    function integerToString(uint _i) internal pure
    returns (string memory) {
        if (_i == 0) {
            return "0";
        }
        uint j=0;
        uint len;
        for (j = _i; j != 0; j /= 10) { //for loop example
            len++;
        }
        bytes memory bstr = new bytes(len);
        uint k = len - 1;
        while (_i != 0) {
            bstr[k--] = byte(uint8(48 + _i % 10));
            _i /= 10;
        }
        return string(bstr); //access local variable
    }
}
```

4.loop Control: (Break statement)

```
pragma solidity ^0.5.0;

contract SolidityTest {
    uint storedData;
    constructor() public {
        storedData = 10;
    }
    function getResult() public view returns(string memory){
```



```
uint a = 1;
uint b = 2;
uint result = a + b;
return integerToString(result);
}
function integerToString(uint _i) internal pure
returns (string memory) {

    if(_i == 0) {
        return "0";
    }
    uint j = _i;
    uint len;

    while (true) {
        len++;
        j /= 10;
        if(j==0){
            break; //using break statement
        }
    }
    bytes memory bstr = new bytes(len);
    uint k = len - 1;

    while (_i != 0) {
        bstr[k--] = byte(uint8(48 + _i % 10));
        _i /= 10;
    }
    return string(bstr);
}
}
```

(continue statement)

```
pragma solidity ^0.5.0;
contract SolidityTest {
    uint storedData;
    constructor() public{
        storedData = 10;
    }
    function getResult() public view returns(string memory){
        uint n = 1;
        uint sum = 0;
```

```
while( n < 10){
n++;
if(n == 5){
continue; // skip n in sum when it is 5.
}
sum = sum + n;
}
return integerToString(sum);
}
function integerToString(uint _i) internal pure
returns (string memory) {

if (_i == 0) {
return "0";
}
uint j = _i;
uint len;

while (true) {
len++;
j /= 10;
if(j==0){
break; //using break statement
}
}
bytes memory bstr = new bytes(len);
uint k = len - 1;

while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);
}
}
```

D) Decision Making:

While writing a program, there may be a situation when you need to adopt one out of a given set of paths. In such cases, you need to use conditional statements that allow your program to make correct decisions and perform right actions. Solidity supports conditional statements which are used to perform different actions based on different conditions. Here we will explain the **if..else** statement.

1.if statement: The **if** statement is the fundamental control statement that allows Solidity to make decisions and execute statements conditionally.

```
pragma solidity ^0.5.0;

contract SolidityTest {
    uint storedData;
    constructor() public {
        storedData = 10;
    }
    function getResult() public view returns(string memory){
        uint a = 1;
        uint b = 2;
        uint result = a + b;
        return integerToString(result);
    }
    function integerToString(uint _i) internal pure
    returns (string memory) {
        if (_i == 0) { // if statement
            return "0";
        }
        uint j = _i;
        uint len;

        while (j != 0) {
            len++;
            j /= 10;
        }
        bytes memory bstr = new bytes(len);
        uint k = len - 1;

        while (_i != 0) {
            bstr[k--] = byte(uint8(48 + _i % 10));
            _i /= 10;
        }
    }
}
```

```
return string(bstr);//access local variable
}}
```

2.if-else statement: The 'if...else' statement is the next form of control statement that allows Solidity to execute statements in a more controlled way.

```
pragma solidity ^0.5.0;

// Creating a contract
contract Types {
// Declaring state variables
uint i = 10;
bool even;

// Defining function to
// demonstrate the use of
// 'if...else statement'
function decision_making(
) public payable returns(bool){
if(i%2 == 0){
even = true;
}
else{
even = false;
}
return even;
}
}
```

3.if-else..if statement: The if...else if... statement is an advanced form of if...else that allows Solidity to make a correct decision out of several conditions.

```
pragma solidity ^0.5.0;

// Creating a contract
contract Types {
// Declaring state variables
uint i = 12;
string result;
// Defining function to
// demonstrate the use
// of 'if...else if...else
```

```
// statement'  
function decision_making (  
) public returns(string memory){  
    if(i<10){  
        result = "less than 10";  
    }  
    else if(i == 10){  
        result = "equal to 10";  
    }  
    else{  
        result = "greater than 10";  
    }  
    return result;  
}  
}
```

E) String:

// Solidity program to demonstrate

// how to create a contract

```
pragma solidity ^0.4.23;  
  
// Creating a contract  
contract Test {  
    // Declaring variable  
    string str;  
  
    // Defining a constructor  
    constructor(string str_in){  
        str = str_in;  
    }  
    // Defining a function to  
    // return value of variable 'str'  
    function str_out() public view returns(string memory){  
        return str;  
    }  
}
```

Note: after deploy it asked u to enter string then enter string over there and then see the output after clicking on str_out button

F) String:

Solidity supports String literal using both double quote (") and single quote ('). It provides string as a data type to declare a variable of type String.(Int to str)

```
pragma solidity ^0.5.0;

contract SolidityTest {
    constructor() public {
    }
    function getResult() public view returns(string memory){
        uint a = 1;
        uint b = 2;
        uint result = a + b;
        return integerToString(result);
    }
    function integerToString(uint _i) internal pure
    returns (string memory) {

        if(_i == 0) {
            return "0";
        }
        uint j = _i;
        uint len;

        while (j != 0) {
            len++;
            j /= 10;
        }
        bytes memory bstr = new bytes(len);
        uint k = len - 1;

        while (_i != 0) {
            bstr[k--] = byte(uint8(48 + _i % 10));
            _i /= 10;
        }
        return string(bstr);
    }
}
```

B)Array:

Array is a data structure, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

// Solidity program to demonstrate

// accessing elements of an array

```
pragma solidity ^0.5.0;

// Creating a contract
contract Types {

// Declaring an array
uint[6] data;
uint x;

// Defining function to
// assign values to array
function array_example() public returns (uint[6] memory)
{

data = [uint(10), 20, 30, 40, 50, 60];
}
function result() public view returns(uint[6] memory){
return data;
}
// Defining function to access
// values from the array
// from a specific index
function array_element() public view returns (uint){
uint x = data[2];
return x;
}
}
```

C)Enums:

Enums restrict a variable to have one of only a few predefined values. The values in this enumerated list are called enums. With the use of enums it is possible to reduce the number of bugs in your code.

// Solidity program to demonstrate

// how to use 'enumerator'

```
pragma solidity ^0.5.0;

// Creating a contract
contract Types {

// Creating an enumerator
enum week_days
{
Monday,
Tuesday,
Wednesday,
Thursday,
Friday,
Saturday,
Sunday
}

// Declaring variables of
// type enumerator
week_days week;

week_days choice;

// Setting a default value
week_days constant default_value
= week_days.Sunday;

// Defining a function to
// set value of choice
function set_value() public {
choice = week_days.Thursday;
}

// Defining a function to
// return value of choice
function get_choice(
) public view returns (week_days) {
return choice;
}

// Defining function to
// return default value
function getdefaultvalue(
) public pure returns(week_days) {
return default_value;
}
}
```


G)Structure:

Struct types are used to represent a record.

```
pragma solidity ^0.5.0;

contract test {
  struct Book {
    string title;
    string author;
    uint book_id;
  }
  Book book;

  function setBook() public {
    book = Book('Learn Java', 'TP', 1);
  }
  function getBookId() public view returns (uint) {
    return book.book_id;
  }
}
```

H) Mappings:

Mapping is a reference type as arrays and structs. Following is the syntax to declare a mapping type.

mapping(_KeyType => _ValueType) where ,

_KeyType – can be any built-in types plus bytes and string. No reference type or complex objects are allowed.

_ValueType – can be any type.

```
pragma solidity ^0.5.0;

contract LedgerBalance {
  mapping(address => uint) balance;

  function updateBalance() public returns(uint) {
```

```
balance[msg.sender]=30;  
return balance[msg.sender];  
}  
}
```

Mapping program for String.

```
pragma solidity ^0.5.0;  
  
contract LedgerBalance {  
mapping(address => string) name;  
  
function updateBalance() public returns(string memory){  
name[msg.sender] = "Mrunali";  
return name[msg.sender];  
}  
function printsender() public view returns(address) {  
return msg.sender;  
}  
}
```

I) CONVERSION:

```
// SPDX-License-Identifier: GPL-3.0  
pragma solidity 0.8.12;  
  
contract Conversion  
{  
    uint a=12;  
    uint8 b=14;  
    uint16 c=16;  
  
    function add() public view returns (uint)  
    {  
        uint d = a + uint(b) + uint(c);  
        return d;  
    }  
}
```

J) EtherUnit:

```
// SPDX-License-Identifier: GPL-3.0  
pragma solidity 0.5.13;
```

```
contract EtherUnits
{
    function units() public pure returns (bool,bool,bool)
    {
        bool a=false;
        bool b=false;
        bool c=false;

        if (1 ether == 1000 finney)
            a=true;
        if (1 ether == 1000000 szabo)
            b=true;
        if (1 ether == 1000000000000000000000000 wei)
            c=true;

        return (a,b,c);
    }
}
```

K) SpecialVariables:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.12;

contract SpecialVariables
{
    uint data1=0;
    uint data2=0;
    uint data3=0;

    function set() public
    {
        data1=block.difficulty;
        data2=block.timestamp;
        data3=block.number;
    }

    function get() public view returns (uint, uint, uint)
    {
        return(data1, data2, data3);
    }
}
```


PRACTICAL NO. 3B

AIM: WRITE A SOLIDITY PROGRAM FOR FUNCTION, VIEW FUNCTION, PURE

FUNCTION, FALLBACK FUNCTION, FUNCTION OVERLOADING, MATHEMATICAL FUNCTION & CRYPTOGRAPHIC FUNCTIONS.

A)Function:

A function is a group of reusable code which can be called anywhere in your program. This eliminates the need of writing the same code again and again. It helps programmers in writing modular codes. Functions allow a programmer to divide a big program into a number of small and manageable functions.

```
pragma solidity ^0.5.0;

contract SolidityTest {
    constructor() public {
    }
    function getResult() public view returns(string memory){
        uint a = 1;
        uint b = 2;
        uint result = a + b;
        return integerToString(result);
    }
    function integerToString(uint _i) internal pure
    returns (string memory) {

        if(_i == 0) {
            return "0";
        }
        uint j = _i;
        uint len;

        while (j != 0) {
            len++;
            j /= 10;
        }
        bytes memory bstr = new bytes(len);
        uint k = len - 1;

        while (_i != 0) {
            bstr[k--] = byte(uint8(48 + _i % 10));
```

```
_i /= 10;  
}  
return string(bstr); //access local variable  
  
}  
}
```

B)View Function:

View functions ensure that they will not modify the state. A function can be declared as **view**. Getter methods are by default view functions.

```
pragma solidity ^0.5.0;  
contract Test {  
    function getResult() public view returns(uint product, uint sum){  
        uint a = 1; // local variable  
        uint b = 2;  
        product = a * b;  
        sum = a + b;  
    }  
}
```

C)Pure Function:

Pure functions ensure that they not read or modify the state. A function can be declared as **pure**. Pure functions can use the `revert()` and `require()` functions to revert potential state changes if an error occurs.

```
pragma solidity ^0.5.0;  
  
contract Test {  
    function getResult() public pure returns(uint product, uint sum){  
        uint a = 1;  
        uint b = 2;  
        product = a * b;  
        sum = a + b;  
    }  
}
```

D)Fallback Function:

Fallback function is a special function available to a contract.

```
pragma solidity ^0.5.0;
```

```
contract Test {
uint public x ;
function() external { x = 1; }
}
contract Sink {
function() external payable { }
}
contract Caller {
function callTest(Test test) public returns (bool) {
(bool success,) = address(test).call(abi.encodeWithSignature("nonExistingFunction()"));
require(success);
// test.x is now 1
address payable testPayable = address(uint160(address(test)));
// Sending ether to Test contract,
// the transfer will fail, i.e. this returns false here.
return (testPayable.send(2 ether));
}
function callSink(Sink sink) public returns (bool) {
address payable sinkPayable = address(sink);
return (sinkPayable.send(2 ether));
}
}
```

Function Overloading:

The definition of the function must differ from each other by the types and/or the number of arguments in the argument list. You cannot overload function declarations that differ only by return type.

```
pragma solidity ^0.5.0;

contract Test {
    function getSum(uint a, uint b) public pure returns(uint){
        return a + b;
    }
    function getSum(uint a, uint b, uint c ) public pure returns(uint){
        return a + b + c;
    }
    function callSumWithTwoArguments() public pure returns(uint){
        return getSum(2,2);
    }
    function callSumWithThreeArguments() public pure returns(uint){
        return getSum(1,2,4);
    }
}
```

Mathematical Function:

Solidity provides inbuilt mathematical functions as well.

```
pragma solidity ^0.5.0;

contract Test {
    function callAddMod() public pure returns(uint){
        return addmod(4, 5, 3);
    }
    function callMulMod() public pure returns(uint){
        return mulmod(4, 5, 3);
    }
}
```

Cryptographic Function:

Solidity provides inbuilt cryptographic functions as well.

```
pragma solidity ^0.5.0;
```



```
contract Test {  
function callKeccak256() public pure returns(bytes32 result){  
return keccak256("ABC");  
}  
}
```

PRACTICAL NO. 4B

AIM: WRITE A SOLIDITY PROGRAM FOR CONTRACT, INHERITANCE, CONSTRUCTORS, ABSTRACT CONTRACTS, INTERFACES.

A)Contract:

Contract in Solidity is similar to a Class in C++. A Contract have following properties.

Constructor – A special function declared with constructor keyword which will be executed once per contract and is invoked when a contract is created.

State Variables – Variables per Contract to store the state of the contract.

Functions – Functions per Contract which can modify the state variables to alter the state of a contract.

// Calling function from external contract

```
pragma solidity ^0.5.0;
contract C {
//private state variable
uint private data;

//public state variable
uint public info;

//constructor
constructor() public {
info = 10;
}
//private function
function increment(uint a) private pure returns(uint) { return a + 1; }
//public function
function updateData(uint a) public { data = a; }
function getData() public view returns(uint) { return data; }
function compute(uint a, uint b) internal pure returns (uint) { return a + b; }
}
//Derived Contract
contract E is C {
uint private result;
C private c;
```

```
constructor() public {  
  c = new C();  
}  
function getComputedResult() public {  
  result = compute(3, 5);  
}  
function getResult() public view returns(uint) { return result; }  
function getData() public view returns(uint) { return c.info(); }  
}
```

B)Inheritance:

Inheritance is a way to extend functionality of a contract. Solidity supports both single as well as multiple inheritance.

// Solidity program to

// demonstrate

// Single Inheritance

```
pragma solidity >=0.4.22 <0.6.0;
```

// Defining contract

```
contract parent{
```

// Declaring internal

// state variable

```
uint internal sum;
```

// Defining external function

// to set value of internal

// state variable sum

```
function setValue() external {
```

```
uint a = 20;
```

```
uint b = 20;
```

```
sum = a + b;
```

```
}
```

```
}
```

// Defining child contract

```
contract child is parent{
```

// Defining external function

// to return value of

```
// internal state variable sum
function getValue() external view returns(uint) {
return sum;
}
}
// Defining calling contract
contract caller {

// Creating child contract object
child cc = new child();

// Defining function to call
// setValue and getValue functions
function testInheritance() public {
cc.setValue();
}
function result() public view returns(uint ){
return cc.getValue();
}
}
```

C)Constructors:

Constructor is a special function declared using constructor keyword. It is an optional function and is used to initialize state variables of a contract. Following are the key characteristics of a constructor.

A contract can have only one constructor.

A constructor code is executed once when a contract is created and it is used to initialize contract state.

A constructor can be either public or internal.

An internal constructor marks the contract as abstract.

In case, no constructor is defined, a default constructor is present in the contract.

```
pragma solidity ^0.5.0;
contract Base {
uint data;
constructor(uint _data) public {
data = _data;
}
function getResult()public view returns(uint){
```

```
return data;
}
}
contract Derived is Base (5) {
constructor() public {}
}
```

// Indirect Initialization of Base Constructor

```
pragma solidity ^0.5.0;

contract Base {
uint data;
constructor(uint _data) public {
data = _data;
}
function getResult() public view returns(uint){
return data;
}
}
contract Derived is Base {
constructor(uint _info) Base(_info * _info) public {}
}
```

D)Abstract Contracts:

Abstract Contract is one which contains at least one function without any implementation. Such a contract is used as a base contract. Generally an abstract contract contains both implemented as well as abstract functions. Derived contract will implement the abstract function and use the existing functions as and when required.

```
pragma solidity ^0.5.0;

contract Calculator {
function getResult() public view returns(uint);
}
contract Test is Calculator {
function getResult() public view returns(uint) {
uint a = 4;
uint b = 2;
```

```
uint result = a + b;  
return result;  
}  
}
```

E)Interfaces:

Interfaces are similar to abstract contracts and are created using interface keyword. Following are the key characteristics of an interface.

Interface can not have any function with implementation.

Functions of an interface can be only of type external.

Interface can not have constructor.

Interface can not have state variables.

```
pragma solidity ^0.5.0;  
  
interface Calculator {  
    function getResult() external view returns(uint);  
}  
  
contract Test is Calculator {  
    constructor() public {}  
    function getResult() external view returns(uint){  
        uint a = 5;  
        uint b = 2;  
        uint result = a + b;  
        return result;  
    }  
}
```

PRACTICAL NO. 6

AIM: Write a program to demonstrate mining of Ether.

Code:

```
import hashlib

def sha256(message):

    return hashlib.sha256(message.encode('ascii')).hexdigest()


def mine(message, difficulty=1):

    assert difficulty >= 1

    prefix = '1' * difficulty

    for i in range(1000):

        digest = sha256(str(hash(message)) + str(i))

        if digest.startswith(prefix):

            print ("after " + str(i) + " iterations found nonce: " + digest)

    return digest


mine ("test message", 2)
```

PRACTICAL NO. 7**AIM: Demonstrate the running of the blockchain node.****Code:**

For timestamp

import datetime

Calculating the hash

in order to add digital

fingerprints to the blocks

import hashlib

To store data

in our blockchain

import json

Flask is for creating the web

app and jsonify is for

displaying the blockchain

from flask import Flask, jsonify

class Blockchain:

This function is created


```
# to create the very first
# block and set its hash to "0"
def __init__(self):
    self.chain = []
    self.create_block(proof=1, previous_hash='0')

# This function is created
# to add further blocks
# into the chain
def create_block(self, proof, previous_hash):
    block = {'index': len(self.chain) + 1,
             'timestamp': str(datetime.datetime.now()),
             'proof': proof,
             'previous_hash': previous_hash}
    self.chain.append(block)
    return block

# This function is created
# to display the previous block
def print_previous_block(self):
    return self.chain[-1]

# This is the function for proof of work
# and used to successfully mine the block
```

```
def proof_of_work(self, previous_proof):  
    new_proof = 1  
    check_proof = False  
  
    while check_proof is False:  
        hash_operation = hashlib.sha256(  
            str(new_proof**2 -  
previous_proof**2).encode()).hexdigest()  
        if hash_operation[:5] == '00000':  
            check_proof = True  
        else:  
            new_proof += 1  
  
    return new_proof  
  
def hash(self, block):  
    encoded_block = json.dumps(block, sort_keys=True).encode()  
    return hashlib.sha256(encoded_block).hexdigest()  
  
def chain_valid(self, chain):  
    previous_block = chain[0]  
    block_index = 1  
  
    while block_index < len(chain):  
        block = chain[block_index]
```

```
        if block['previous_hash'] != self.hash(previous_block):  
            return False  
  
        previous_proof = previous_block['proof']  
        proof = block['proof']  
        hash_operation = hashlib.sha256(  
            str(proof**2 -  
previous_proof**2).encode()).hexdigest()  
  
        if hash_operation[:5] != '00000':  
            return False  
  
        previous_block = block  
        block_index += 1  
  
    return True
```

```
# Creating the Web
```

```
# App using flask
```

```
app = Flask(__name__)
```

```
# Create the object
```

```
# of the class blockchain
```

```
blockchain = Blockchain()
```

Mining a new block

```
@app.route('/mine_block', methods=['GET'])
```

```
def mine_block():
```

```
    previous_block = blockchain.print_previous_block()
```

```
    previous_proof = previous_block['proof']
```

```
    proof = blockchain.proof_of_work(previous_proof)
```

```
    previous_hash = blockchain.hash(previous_block)
```

```
    block = blockchain.create_block(proof, previous_hash)
```

```
    response = {'message': 'A block is MINED',
```

```
                'index': block['index'],
```

```
                'timestamp': block['timestamp'],
```

```
                'proof': block['proof'],
```

```
                'previous_hash': block['previous_hash']}]
```

```
    return jsonify(response), 200
```

Display blockchain in json format

```
@app.route('/get_chain', methods=['GET'])
```

```
def display_chain():
```

```
response = {'chain': blockchain.chain,  
            'length': len(blockchain.chain)}  
  
return jsonify(response), 200
```

Check validity of blockchain

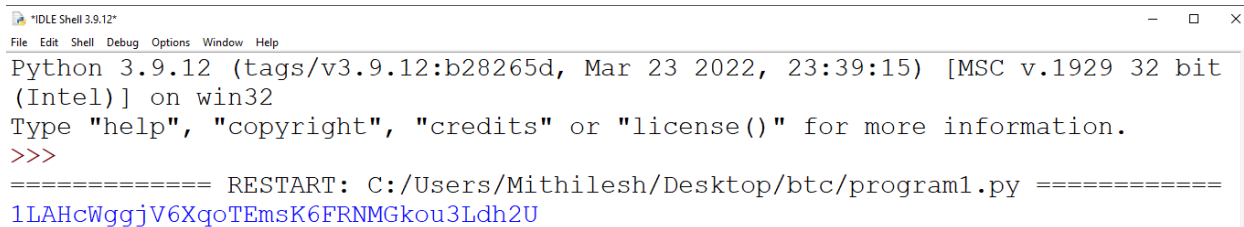
```
@app.route('/valid', methods=['GET'])  
def valid():  
    valid = blockchain.chain_valid(blockchain.chain)  
  
    if valid:  
        response = {'message': 'The Blockchain is valid.'}  
    else:  
        response = {'message': 'The Blockchain is not valid.'}  
  
    return jsonify(response), 200
```

Run the flask server locally

```
app.run(host='127.0.0.1', port=5000)
```

PRACTICAL NO. 8**AIM: Demonstrate the use of Bitcoin Core API.****CODE:**

```
#pip install bitcoinlib
from bitcoinlib.wallets import Wallet
w = Wallet.create('Wallet1')
key1 = w.get_key()
print(key1.address)
w.scan()
print(w.info())
```

A screenshot of a Python IDLE Shell window. The title bar reads "IDLE Shell 3.9.12*". The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The shell displays the following text: "Python 3.9.12 (tags/v3.9.12:b28265d, Mar 23 2022, 23:39:15) [MSC v.1929 32 bit (Intel)] on win32", "Type 'help', 'copyright', 'credits' or 'license()' for more information.", and a prompt ">>>". Below the prompt, a restart message is shown: "===== RESTART: C:/Users/Mithilesh/Desktop/btc/program1.py =====". The final output is a long alphanumeric string: "1LAHcWggjV6XqoTEmsK6FRNMGkou3Ldh2U".

```
*IDLE Shell 3.9.12*
File Edit Shell Debug Options Window Help
Python 3.9.12 (tags/v3.9.12:b28265d, Mar 23 2022, 23:39:15) [MSC v.1929 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Mithilesh/Desktop/btc/program1.py =====
1LAHcWggjV6XqoTEmsK6FRNMGkou3Ldh2U
```

PRACTICAL NO. 9**AIM: Create your own blockchain and demonstrate its use.**

CODE:

```
# -*- coding: utf-8 -*-
```

```
"""BC_7B.ipynb
```

Automatically generated by Colaboratory.

Original file is located at

<https://colab.research.google.com/drive/1-U7Nm2XGwjltPRCkqT52OuF6SfBONiQb>

```
"""
```

```
import collections
```

```
import datetime
```

```
import binascii
```

```
!pip install pycryptodome
```

```
import Crypto
```

```
from Crypto.PublicKey import RSA
```

```
from Crypto import Random
```

```
from Crypto.Hash import SHA
```

```
from Crypto.Signature import PKCS1_v1_5
```

```
class Client:
```

```
    def __init__(self):
```

```
        random = Crypto.Random.new().read
```

```
        self._private_key = RSA.generate(1024, random)
```

```
        self._public_key = self._private_key.publickey()
```

```
        self._signer = PKCS1_v1_5.new(self._private_key)
```

```
@property
```

```
def identity(self):
```

```
    return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
```

```
class Transaction:
```

```
    def __init__(self, sender, recipient, value):
```

```
        self.sender = sender
```

```
        self.recipient = recipient
```

```
        self.value = value
```

```
        self.time = datetime.datetime.now()
```

```
    def to_dict(self):
```

```
        if self.sender == "Genesis":
```

```
            identity = "Genesis"
```

```
        else:
```

```
            identity = self.sender.identity
```

```
    return collections.OrderedDict({
```

```
        'sender': identity,
```

```
        'recipient': self.recipient,
```

```
        'value': self.value,
```

```
        'time' : self.time})
```

```
    def sign_transaction(self):
```

```
        private_key = self.sender._private_key
```

```
        signer = PKCS1_v1_5.new(private_key)
```



```
h = SHA.new(str(self.to_dict()).encode('utf8'))
return binascii.hexlify(signer.sign(h)).decode('ascii')
```

```
import hashlib
```

```
def sha256(message):
    return hashlib.sha256(message.encode('ascii')).hexdigest()
```

```
def mine(message, difficulty=1):
    assert difficulty >= 1
    prefix = '1' * difficulty
    for i in range(1000):
        digest = sha256(str(hash(message)) + str(i))
        if digest.startswith(prefix):
            print ("after " + str(i) + " iterations found nonce: " + digest)
    return digest
```

```
class Block:
```

```
    def __init__(self):
        self.verified_transactions = []
        self.previous_block_hash = ""
        self.Nonce = ""
```

```
last_block_hash = ""
```

```
def display_transaction(transaction):
```

```
    #for transaction in transactions:
```

```
dict = transaction.to_dict()
print ("sender: " + dict['sender'])
print ('-----')
print ("recipient: " + dict['recipient'])
print ('-----')
print ("value: " + str(dict['value']))
print ('-----')
print ("time: " + str(dict['time']))
print ('-----')
```

```
TPCoins = []
```

```
def dump_blockchain (self):
```

```
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')
```

```
last_transaction_index = 0
```

```
transactions = []
```

```
Raja = Client()
```

```
Rani = Client()
```

```
Seema = Client()
```

```
Reema = Client()
```

```
t1 = Transaction(Raja, Rani.identity, 15.0)
```

```
t1.sign_transaction()
```

```
transactions.append(t1)
```

```
t2 = Transaction(Raja, Seema.identity, 6.0)
```

```
t2.sign_transaction()
```

```
transactions.append(t2)
```

```
t3 = Transaction(Rani, Reema.identity, 2.0)
```

```
t3.sign_transaction()
```

```
transactions.append(t3)
```

```
t4 = Transaction(Seema, Rani.identity, 4.0)
```

```
t4.sign_transaction()
```

```
transactions.append(t4)
```

```
t5 = Transaction(Reema, Seema.identity, 7.0)
```

```
t5.sign_transaction()
```

```
transactions.append(t5)
```

```
t6 = Transaction(Rani, Seema.identity, 3.0)
```

```
t6.sign_transaction()
```

```
transactions.append(t6)
```

```
t7 = Transaction(Seema, Raja.identity, 8.0)
```

```
t7.sign_transaction()
```

```
transactions.append(t7)
```

```
t8 = Transaction(Seema,Rani.identity,1.0)
```

```
t8.sign_transaction()
```

```
transactions.append(t8)
```

```
t9 = Transaction(Reema,Raja.identity,5.0)
```

```
t9.sign_transaction()
```

```
transactions.append(t9)
```

```
t10 = Transaction(Reema,Rani.identity,3.0)
```

```
t10.sign_transaction()
```

```
transactions.append(t10)
```

```
# Miner 1 adds a block
```

```
block = Block()
```

```
for i in range(3):
```

```
    temp_transaction = transactions[last_transaction_index]
```

```
    # validate transaction
```

```
    # if valid
```

```
    block.verified_transactions.append (temp_transaction)
```

```
    last_transaction_index += 1
```

```
block.previous_block_hash = last_block_hash
```

```
block.Nonce = mine (block, 2)
```

```
digest = hash (block)
```

```
TPCoins.append (block)
```

```
last_block_hash = digest
```

```
# Miner 2 adds a block
```

```
block = Block()
```

```
for i in range(3):
```

```
    temp_transaction = transactions[last_transaction_index]
```

```
    # validate transaction
```

```
    # if valid
```

```
    block.verified_transactions.append (temp_transaction)
```

```
    last_transaction_index += 1
```

```
block.previous_block_hash = last_block_hash
```

```
block.Nonce = mine (block, 2)
```

```
digest = hash (block)
```

```
TPCoins.append (block)
```

```
last_block_hash = digest
```

```
# Miner 3 adds a block
```

```
block = Block()
```

```
for i in range(3):
```

```
    temp_transaction = transactions[last_transaction_index]
```

```
    # validate transaction
```

```
    # if valid
```

```
    block.verified_transactions.append (temp_transaction)
```

```
last_transaction_index += 1

block.previous_block_hash = last_block_hash
block.Nonce = mine (block, 2)
digest = hash (block)
TPCoins.append (block)
last_block_hash = digest

dump_blockchain(TPCoins)
```

PRACTICAL NO. 10

AIM: Build Dapps with angular.

CODE:

```
pragma solidity 0.8.13;
```

```
contract HelloWorld
```

```
{
```

```
    string greet;
```

```
    constructor()
```

```
    {
```

```
        greet="Hello World";
```

```
    }
```

```
    function getgreet() public view returns(string memory)
```

```
    {
```

```
        return greet;
```

```
    }
```

```
    function setgreet(string memory greet1) public
```

```
    {
```

```
        greet=greet1
```

```
    }
```

```
}
```